CiMFlowSim: A Dataflow Simulator for Compute-in-Memory Accelerators

Junsu Heo, Sungkyung Park, *Senior Member, IEEE,* and Chester Sungchung Park, *Senior Member, IEEE*

***Abstract*—Compute-in-memory (CiM) is a widely adopted approach for energy-efficient deep neural network (DNN) acceleration. The latency, energy, and hardware area of a CiM accelerator are largely determined by the dataflow across its heterogeneous intellectual property (IP) blocks. Therefore, the design space exploration (DSE) of dataflow is essential to the design of CiM accelerators. However, the conventional DSE approaches may fail to approach the Pareto front owing to inaccuracy of the cost models. This paper proposes CiMFlowSim, a cycle-level dataflow simulator for CiM accelerators, which is characterized by its dependency-aware pipeline modeling and data-layout-aware memory modeling. The decoupled loop tiling, which allows the input and output tile sizes to be configured independently, is newly considered for the DSE of the weight-stationary CiM pipeline. Experimental results show that the estimation error of the analytical cost model amounts to 41.1% in latency and 56.9% in energy. When applied to loop tiling optimization, CiMFlowSim helps improve the Pareto front by up to 153.1% in terms of hypervolume (HV). In addition, the inclusion of decoupled loop tiling into the design space improves the Pareto front by up to 63.7%.**

***Index Terms*—Compute-in-memory (CiM), convolutional neural network (CNN), design space exploration, loop tiling, data layout.**

I. Introduction

COMPUTE-in-memory (CiM) has emerged as a promising approach for overcoming the von Neumann bottleneck in deep neural network (DNN) acceleration [2]–[5]. Among CiM technologies, non-volatile memory (NVM)-based implementations using resistive RAM (ReRAM), magnetoresistive RAM (MRAM), and embedded flash (eFlash) are favored thanks to their high energy efficiency [5]–[10]. NVM-based CiM accelerators often adopt a weight-stationary dataflow, in which all the weights remain stationary in convolution layers [7], [11], [12].

The latency, energy, and hardware area of a CiM accelerator are heavily dependent on the dataflow across its heterogeneous intellectual property (IP) blocks such as on-chip buffer and CiM array, in particular, loop ordering and tiling [11], [12]. Therefore, design space exploration (DSE) of the dataflow is essential for optimizing CiM accelerators [13]–[15]. As illustrated in Fig. 1(a), the corresponding DSE generally consists of three components: search algorithm, design space, and cost model. The search algorithm selects one of the design points in the design space, and then the latency, energy, and hardware area of the selected design point are estimated by the cost model. The search algorithm primarily affects exploration efficiency whereas the design space and the cost model impact the optimality of the final Pareto front (given a sufficient exploration time) [16]. When it comes to the design space for NVM-based CiM accelerators, assuming weight-stationary dataflow heavily constrains loop ordering, leaving loop tiling as a major dataflow parameter [17].

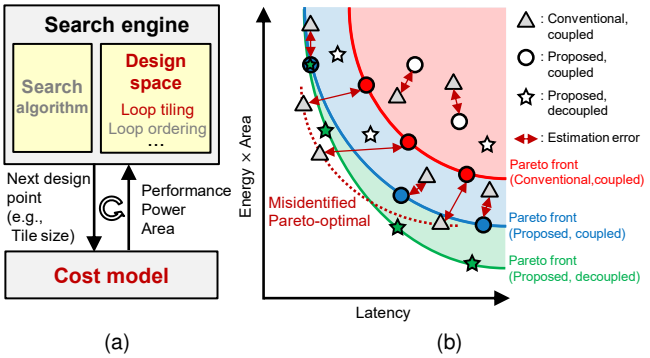


Fig. 1. (a) Dataflow DSE process. (b) Impact of cost model (red vs. blue) and design space (blue vs. green).

Existing DSE approaches [1], [18]–[23] remain limited in accuracy of the cost model as well as in coverage of the design space. First, the conventional cost models tend to ignore either inter-stage pipeline stalls or memory data layout overheads, resulting in inaccurate latency/energy estimation, which may severely mislead the corresponding DSE. As illustrated in Fig. 1(b), an inaccurate cost model may misidentify non-optimal design points as Pareto-optimal (red dashed line), whereas their actual latency and energy–area product (EAP) lie in the red region. Second, in most of the existing approaches the design space is only confined to coupled loop tiling in which the input tile size is derived from the output tile size. As indicated by the blue region in Fig. 1(b), the limited coverage of design space through the exclusion of decoupled loop tiling may also cause the corresponding DSE to end up in non-optimal design points.

To address these limitations, the following two contributions are made in this paper:

A preliminary version of this work was presented at the Design, Automation and Test in Europe Conference (DATE) 2023 [1]. This manuscript extends it with a dependency-aware pipeline model, a data-layout-aware memory model, a generalized decoupled-tiling formulation, and an expanded evaluation across multiple hardware configurations and CNN benchmarks. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00222085, Development of memory module and memory compiler for non-volatile PIM optimized for data characteristics and data access characteristics of AI processor). The source code is available at https://github.com/CiMFlowSim. *(Corresponding author: Chester Sungchung Park.)*

Junsu Heo and Chester Sungchung Park are with Konkuk University, Seoul, South Korea (e-mail: junsuheo@konkuk.ac.kr; chester@konkuk.ac.kr).

Sungkyung Park is with Pusan National University, Busan, South Korea (e-mail: fspark@pusan.ac.kr).

1) **CiMFlowSim:** A cycle-level dataflow simulator for weight-stationary CiM accelerators. The SystemC/transaction-level modeling (TLM) simulator proposes both a dependency-aware pipeline model and a data-layout-aware memory model in order to capture both inter-stage pipeline stalls and memory data layout overheads. This simulator enables cycle-level evaluation of design points whose latency and energy diverge from analytical estimates by up to 41.1% and 56.9%, respectively.
2) **Decoupled loop tiling:** Decoupled loop tiling is newly considered for DSE of the dataflow for weight-stationary CiM accelerators. Also, an enumeration algorithm is provided to define the valid design space for efficient double-buffering execution.

Experimental results evaluate the impact of the aforementioned contributions. CiMFlowSim helps improve the estimation accuracy by up to 41.1% in latency and 56.9% in energy compared to the analytical cost model. When applied to loop tiling optimization, CiMFlowSim improves the hypervolume (HV) [24] by up to 153.1%, where a larger HV indicates broader coverage of the latency–EAP tradeoff space. In addition, the introduction of decoupled loop tiling into the design space further improves the HV by up to 63.7%.

The remainder of this paper is organized as follows. Section II provides the background of this work and the related work. Section III describes decoupled loop tiling for CiM accelerators, and Section IV provides technical details of CiMFlowSim. Section V provides experimental results on latency/energy estimation and loop tiling optimization, and Section VI concludes the paper.

II. Background and Related Work

Prior to discussing the technical details of this paper, this section gives an overview on the system architecture, introduces the conventional loop tiling together with the impact of memory data layout, and provides the related work.

A. System Architecture

Fig. 2(a) shows the block diagram of the target CiM accelerator system. The target accelerator system typically consists of off-chip DRAM storing the activation tensors during inference, on-chip input and output SRAM buffers (IBUF and OBUF) each with double buffering for the consecutive data tiles, and a CiM array whose cells store the weights and perform multiply-and-accumulate (MAC) operations in place. The weights remain stationary in the CiM cells, and only the input and output activations are transferred during convolution layers. Partial sums from the CiM array are combined by an accumulation unit, and a post-processing unit applies the nonlinear activation function. This paper focuses on the dataflow, i.e., the data movement of activations, among the heterogeneous IP blocks with widely varying bandwidth and latency, e.g., the off-chip DRAM, the on-chip buffers and a CiM array. This dataflow is assumed to be controlled by input direct memory access controller (IDMAC), output direct memory access controller (ODMAC), input address-generation

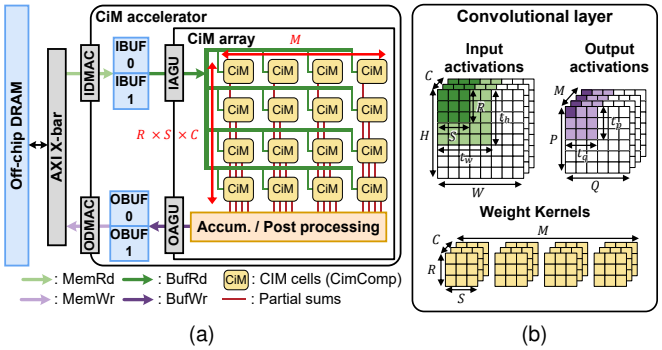


Fig. 2. CiM accelerator architecture. (a) Accelerator block diagram. (b) Variable definitions for activations, weights, and tiling parameters in a convolutional layer.

unit (IAGU), and output address-generation unit (OAGU). In detail, on the input side, the IDMAC and the IAGU transfer input activations from DRAM to the IBUF and from the IBUF to the CiM array, respectively. On the output side, the OAGU and the ODMAC transfer output activations from the CiM array to the OBUF and from the OBUF to DRAM, respectively. Consequently, the overall accelerator system can be seen as a 5-stage pipeline, i.e., consisting of MemRd (Stage 1), BufRd (Stage 2), CiMComp (Stage 3), BufWr (Stage 4), and MemWr (Stage 5): MemRd transfers input activations from DRAM to the IBUF and BufRd does so from the IBUF to the CiM array. Subsequently, CiMComp performs the MAC operation on the CiM array and generates a group of output activations, which is referred to as an output activation (OA) group later in this paper. Finally, BufWr writes the output activations to the OBUF, and MemWr transfers them from the OBUF to DRAM. In order to guarantee functional correctness without any violation of data dependency, both the DMACs and the AGUs should include the synchronization among the heterogeneous IP blocks [25], which will be detailed in the later sections.

Fig. 2(b) defines the parameters for a convolutional layer. The M weight kernels of size $R \times S \times C$ are fully mapped onto the CiM array. The input activation tensor of size $H \times W \times C$ is partitioned into input tiles of size $t_h \times t_w \times C$, and the output activation tensor of size $P \times Q \times M$ is partitioned into output tiles of size $t_p \times t_q \times M$. Here, h and w denote the spatial dimensions of the input, p and q those of the output, and U the convolution stride. As weights are fully mapped, the weight loop is fixed and the tile sizing of input/output activations becomes the principal design parameter for DSE.

B. Coupled Loop Tiling

Although tile sizing is the principal design parameter, conventional loop tiling for DNN accelerators [18], [19] defines the input tile as the region required to compute a given output tile. As a result, the input tile size is derived from the output tile size. Therefore, the two tile sizes are coupled and cannot be adjusted independently, with identical input and output tile sizes. Fig. 3 shows two coupled loop tiling examples for a layer with $H = W = 6$, $P = Q = 4$, $R = S = 3$, $C = 2$, $M = 3$,

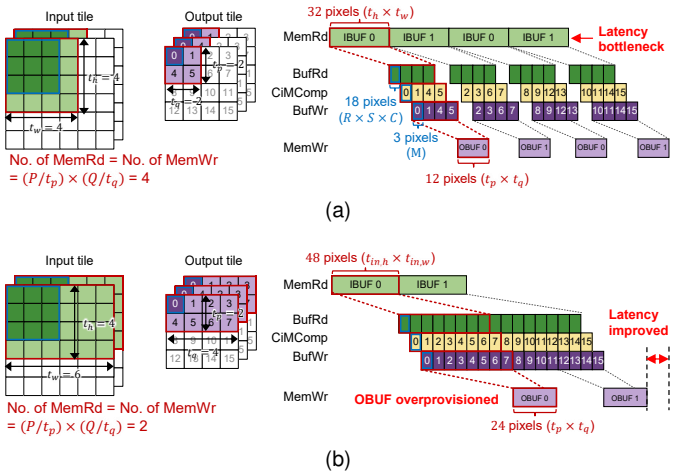


Fig. 3. Coupled loop tiling examples. (a) $(t_p, t_q) = (2, 2)$. (b) $(t_p, t_q) = (2, 4)$.

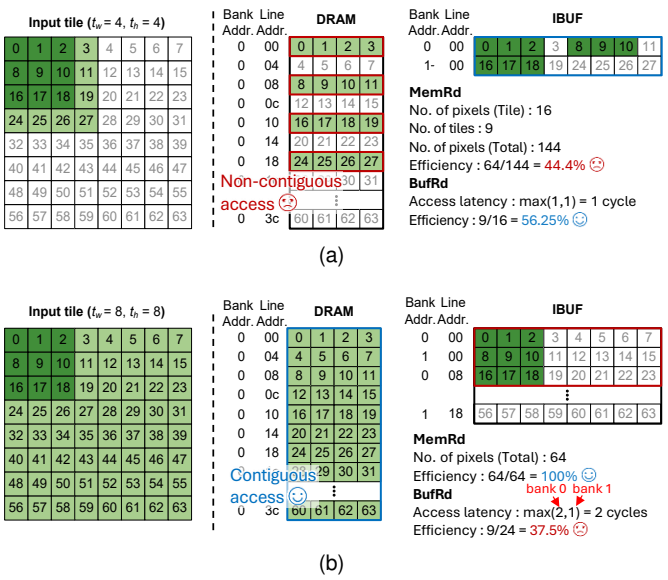


Fig. 4. Data-layout overhead for two tile sizes (3×3 conv., 8×8 input, $C = 1$). (a) Small tile ($t_h = 4$, $t_w = 4$). (b) Large tile ($t_h = 8$, $t_w = 8$).

and $U = 1$. In Fig. 3(a), the output tile size is $(t_p, t_q) = (2, 2)$, and the corresponding input tile size is $(t_h, t_w) = (4, 4)$. Each tile iteration passes through five pipeline stages. MemRd transfers 32 pixels ($t_h \times t_w \times C$) from DRAM to the IBUF, BufRd reads 18 pixels ($R \times S \times C$) from the IBUF to the CiM array for each output pixel, CiMComp performs the MAC operation on the CiM array, BufWr writes an OA group, i.e., 3 output pixels (M) to the OBUF, and MemWr transfers 12 pixels ($t_p \times t_q \times M$) from the OBUF to DRAM. Under the latency-insensitive design (LID) principle [26], each IP block operates independently and communicates through asynchronous handshaking.

MemRd and MemWr operate at the tile level and each iterate $(P/t_p) \times (Q/t_q) = 4$ times. BufRd, CiMComp, and BufWr operate at the output-pixel level and each iterate $P \times Q = 16$ times. As shown in Fig. 3(a), MemRd is the latency bottleneck in this tiling. The MemRd bottleneck can result from redundant reads of overlapping halo regions between adjacent tiles or inefficient memory access patterns owing to memory data layout.

The MemRd bottleneck can be mitigated by increasing the tile size. In Fig. 3(b), the output tile size is increased to $(t_p, t_q) = (2, 4)$. The tile count decreases from 4 to 2, reducing the total number of redundant reads. The IBUF capacity increases from 32 to 48 pixels to accommodate the larger input tile. However, under coupled loop tiling, the OBUF capacity must also increase to 24 pixels, even though the pipeline latency is determined by MemRd, not BufWr, resulting in unnecessary buffer overhead.

C. Impact of Memory Data Layout

The memory data layout defines how each tensor element is placed on the physical memory locations, represented by a tuple of physical indices (e.g., $(\text{Bank}, \text{Row}, \text{Column})$) [27], [28]. The bandwidth/latency and energy consumption of memory access are heavily dependent on the memory data layout.

Fig. 4 illustrates these overheads with a 3×3 convolution on an 8×8 input ($C = 1$) under a channel-last layout, where the channel dimension is stored contiguously [29]. In Fig. 4(a), a 4×4 input tile ($t_h = 4$, $t_w = 4$) requires 9 tiles to cover the output, reading a total of 144 elements owing to halo overlap between adjacent tiles, compared to the 64 unique input elements. Only 44.4% of the data read by MemRd is unique. Each input tile also maps to non-contiguous DRAM addresses, splitting the access into multiple bursts. In Fig. 4(b), the entire 8×8 input ($t_h = 8$, $t_w = 8$) is loaded at once, eliminating halo redundancy with a single contiguous burst. However, the larger tile requires accessing 3 IBUF lines instead of 2, and the additional line access increases the BufRd latency from 1 cycle to 2 cycles. The line utilization also drops from 56.25% to 37.5% as more line contents remain unused. These two overheads, burst fragmentation and line underutilization, are tiling-dependent and are not captured by analytical cost models based on data volume alone.

D. Related Work

Several DSE approaches have been proposed to optimize the dataflow of DNN accelerators. The existing approaches can be mainly categorized by three technical aspects: decoupled loop tiling, dependency-aware pipeline modeling, and data-layout-aware memory modeling.

MAESTRO [18], Timeloop [19], and TileFlow [20] perform dataflow DSE using analytical cost models that provide rapid performance estimation from data access counts. However, none of the aforementioned technical aspects is addressed in these approaches. In order to improve estimation accuracy, a few existing approaches partially address dependency-aware pipeline modeling or data-layout-aware memory modeling. For example, ZigZag [21] models bandwidth-induced stalls, and FEATHER [28] accounts for on-chip data-reordering costs. DNN+NeuroSim [23] provides SPICE-calibrated circuit-level estimation for CiM accelerators, but per-component costs are still aggregated analytically without cycle-level pipeline modeling. More importantly, the design spaces of these approaches are confined to the coupled loop tiling.

Some other approaches broaden the tiling design space by considering decoupled loop tiling. In [22], the tile sizes of the input and output operands along their shared dimension are configured independently for general matrix multiplication (GEMM) on an AMD XDNA2 architecture, but the performance estimation relies on the analytical cost model based on the arithmetic intensity and the bandwidth constraints. In [1], decoupled loop tiling is newly considered for CiM accelerators, but the design space is limited to a few predefined loop tiling options. The absence of dependency-aware pipeline modeling and data-layout-aware memory modeling make it impossible to explore a broad design space of loop tiling for a CiM accelerator with heterogeneous IP blocks.

In summary, no existing approach concurrently addresses all the aforementioned technical aspects, to the best of the authors' knowledge. The limited cost-model accuracy and design-space coverage lead to non-optimal design points in the DSE.

III. Decoupled Loop Tiling

Decoupled loop tiling defines the input and output tile sizes as independent design parameters. This expands the set of input-output tile-size combinations, enabling more flexible tradeoffs among latency, energy, and hardware area. Algorithm 1 shows how to enumerate all the input-output tile-size combinations and filter out some of the less important ones. Loop tiling is applied to (h, w) for the input and (p, q) for the output. Lines 4 and 5 enumerate candidate tuples $(t_{\mathrm{in},p},\ t_{\mathrm{in},q},\ t_{\mathrm{out},p},\ t_{\mathrm{out},q})$ over the divisor sets of P and Q. The input tile size is expressed in output coordinates $(t_{\mathrm{in},p},\ t_{\mathrm{in},q})$ to enable direct ratio comparison with $t_{\mathrm{out},p}$ and $t_{\mathrm{out},q}$ in the filtering conditions, and is converted to input dimensions $t_{\mathrm{in},h}$ and $t_{\mathrm{in},w}$ in Lines 14 and 15. The enumerated candidates are then filtered by two conditions in Lines 8 and 11. Only the remaining candidates are converted into $(t_{\mathrm{in},h},\ t_{\mathrm{in},w})$ and inserted into $\mathcal{T}$ in Lines 14–16. Coupled loop tiling can be seen as a special case where the input and output tile sizes are dependent on each other. In more detail, the enumeration in Algorithm 1 is reduced into the coupled loop tiling by replacing Line 5 with $t_{\mathrm{in},p} = t_{\mathrm{out},p}$ and $t_{\mathrm{in},q} = t_{\mathrm{out},q}$.

The two filtering conditions ensure that the resulting tilings can be executed efficiently with double buffering. First, the condition in Lines 6–9 requires the input-to-output tile count ratio in each dimension to be an integer, i.e., 1:M or N:1. A non-integer ratio causes tile boundaries to misalign with buffer boundaries, preventing full utilization of the buffer capacity at every iteration. Second, the condition in Lines 11–12 excludes the mixed case in which the input tile is larger than the output tile in one dimension but smaller in the other. In such cases, data loaded into the input buffer cannot be fully reused across output tiles, and the same data must be repeatedly read from memory, increasing redundant reads. The valid tilings enumerated by Algorithm 1 define the design space evaluated in this paper.

Fig. 5 illustrates how decoupled loop tiling combines the advantages of both cases in Fig. 3. The input tile covers the same enlarged region as Fig. 3(b), $(t_{\mathrm{in},p}, t_{\mathrm{in},q}) = (2, 4)$,

Algorithm 1 Decoupled loop tiling enumeration

1: **Input:** P, Q (output size), R, S (kernel size), U (stride)
2: **Output:** $\mathcal{T}$ (set of $(t_{\mathrm{out},p}, t_{\mathrm{out},q}, t_{\mathrm{in},h}, t_{\mathrm{in},w})$)
3: $\mathcal{T} \leftarrow \emptyset$
4: **for** $(t_{\mathrm{out},p}, t_{\mathrm{out},q}) \in \mathrm{Divisors}(P) \times \mathrm{Divisors}(Q)$ **do**
5: **for** $(t_{\mathrm{in},p}, t_{\mathrm{in},q}) \in \mathrm{Divisors}(P) \times \mathrm{Divisors}(Q)$ **do**
6: $r_p \leftarrow \max(t_{\mathrm{in},p},\ t_{\mathrm{out},p})\ /\ \min(t_{\mathrm{in},p},\ t_{\mathrm{out},p})$
7: $r_q \leftarrow \max(t_{\mathrm{in},q},\ t_{\mathrm{out},q})\ /\ \min(t_{\mathrm{in},q},\ t_{\mathrm{out},q})$
8: **if** $r_p \notin \mathbb{N}$ **or** $r_q \notin \mathbb{N}$ **then**
9: **continue**
10: **end if**
11: **if** $(t_{\mathrm{in},p} - t_{\mathrm{out},p})(t_{\mathrm{in},q} - t_{\mathrm{out},q}) < 0$ **then**
12: **continue**
13: **end if**
14: $t_{\mathrm{in},h} \leftarrow R + (t_{\mathrm{in},p} - 1) \times U$
15: $t_{\mathrm{in},w} \leftarrow S + (t_{\mathrm{in},q} - 1) \times U$
16: $\mathcal{T} \leftarrow \mathcal{T} \cup \{(t_{\mathrm{out},p}, t_{\mathrm{out},q}, t_{\mathrm{in},h}, t_{\mathrm{in},w})\}$
17: **end for**
18: **end for**
19: **return** $\mathcal{T}$

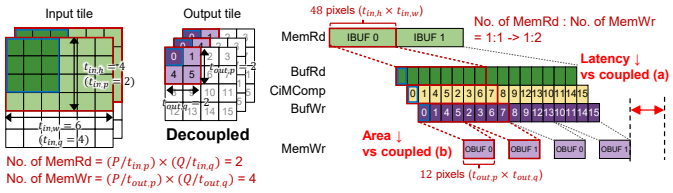


Fig. 5. Decoupled loop tiling example: $(t_{\mathrm{in},p}, t_{\mathrm{in},q}) = (2, 4)$, $(t_{\mathrm{out},p}, t_{\mathrm{out},q}) = (2, 2)$.

reducing MemRd latency through fewer halo reads and better burst efficiency. At the same time, the output tile remains at $(t_{\mathrm{out},p}, t_{\mathrm{out},q}) = (2, 2)$ as in Fig. 3(a), avoiding the 2× OBUF capacity increase. As a result, MemRd iterates 2 times while MemWr iterates 4 times, creating mismatched tile counts across pipeline stages. This mismatch requires the dependency-aware pipeline modeling described in Section IV.

IV. CiMFlowSim

In order to avoid any violation of data dependency among heterogeneous IP blocks (e.g., on-chip buffer and CiM array), the adjacent pipeline stages need to be synchronized to each other. Specifically, according to the producer-consumer scenario [25], a consumer cannot read from a memory location until the producer has written to it (forward dependency). Moreover, a producer cannot write to a memory location until the consumer has read from it (backward dependency). The proposed dependency-aware pipeline modeling makes it possible to guarantee functional correctness over a broad design space of loop tiling, which includes coupled loop tiling as well as decoupled loop tiling. In addition, the bandwidth and latency of an on-chip buffer tend to be highly dependent on the data layout, which motivates us to newly propose the data-layout-aware memory modeling.

CiMFlowSim is written in SystemC/TLM [30], [31], one of the natural choices for dataflow simulation [1], [29], [32]–[35].

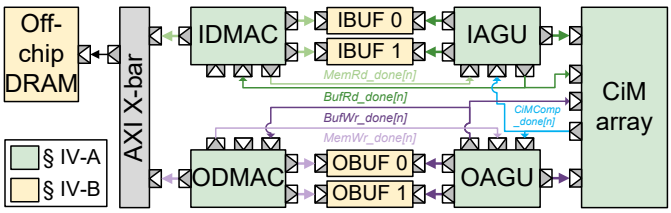


Fig. 6. SystemC/TLM component organization of CiMFlowSim. Bold arrows denote data paths. Thin arrows denote completion signals for inter-stage synchronization. Detailed simulation models are given in Section IV-A (green) and Section IV-B (yellow).

TABLE I
OAGU INTERNAL ELEMENTS

Name	SystemC/TLM Primitive	Role
main_thread	`SC_THREAD`	Control loop over all output activation groups
update_cnt	—	Updates dependency counters on signal arrival
Comp_cnt	—	Completed CiMComp iterations (forward dep.)
MemWr_cnt	—	Completed MemWr iterations (backward dep.)
pl_in/out	`tlm_generic_ payload`	Activation data container
ev_rd/wr	`sc_event`	Read/write completion notification
nb_fw/bw	`nb_transport_fw/bw`	Nonblocking transfer (CiM array→OAGU→OBUF)
t_fw/bw_dep	`target_socket`	Port receiving completion signals
i_in	`initiator_socket`	Port receiving data from CiM array
i_out[0/1]	`initiator_socket`	Port sending data to OBUF (double-buffered)
i_done	`initiator_socket`	Port sending completion to adjacent stages

Fig. 6 shows how CiMFlowSim models the target accelerator system using SystemC/TLM. Each of the IP blocks in Fig. 2(a) is implemented as an `SC_MODULE` and connected to each other through TLM2.0 sockets, a standardized transaction interface introduced in SystemC/TLM. The dependency-aware pipeline modeling is applied to the IP components marked in green, whereas the data-layout-aware memory modeling is applied to those marked in yellow. In addition, the figure includes the completion signals used for inter-stage synchronization (e.g., *BufWr_done*, *MemWr_done*), which will be detailed later in this section. Most of the simulation models of CiMFlowSim (e.g., DMACs, on-chip buffers) were validated extensively against the register transfer level (RTL) implementations in order to prove the cycle accuracy of simulations [1], [29], [32]–[35].

A. Dependency-Aware Pipeline Modeling

The synchronization between adjacent pipeline stages is performed based on the completion signals shared by the pipeline stages. A consumer checks the forward dependency by counting the assertions of the completion signal generated by a producer. Likewise, a producer checks the backward dependency by counting the assertions of the completion signal generated by a consumer. A pipeline stage continues to stall until the assertion count is above a pre-defined threshold specific to each completion signal. This makes it possible to differentiate the synchronization granularity between adjacent pipeline stages. For example, the OAGU checks the forward dependency based on *CiMComp_done* (generated by CiMComp) whereas it checks the backward dependency based on *MemWr_done* (generated by ODMAC). The synchronization with CiMComp is performed on a per-OA-group basis, whereas the one with ODMAC is on a per-output-tile basis. This dependency-aware pipeline modeling makes it possible to guarantee functional correctness over a broad design space of loop tiling including decoupled loop tiling for heterogeneous IP blocks with widely varying bandwidth and latency.

Fig. 7 shows the internal structure of the OAGU as a representative component, and Table I lists its internal elements with the corresponding SystemC/TLM primitives. The main element is main_thread, an `SC_THREAD` that executes the control loop. The main_thread receives completion signals through two `target_socket` ports (t_fw_dep and t_bw_dep), which update the forward counter Comp_cnt and the backward counter MemWr_cnt via the update_cnt callback. Data enters through the `initiator_socket` port i_in from the CiM array and exits through i_out[0] or i_out[1] to the corresponding OBUF bank. The transfers use `tlm_generic_payload` containers (pl_in and pl_out) and `nb_transport_fw/bw` nonblocking transports. The `sc_event` primitives ev_rd and ev_wr notify the main_thread when each transaction completes, and i_done signals completion to the adjacent components.

Fig. 8(a) shows the OAGU thread pseudocode, and Fig. 8(b) shows the resulting dependency pattern for the decoupled tiling in Fig. 5, where the BufWr-to-MemWr ratio is 4:1. The main_thread iterates over all $P \times Q$ output activation groups. CiMComp and BufWr each execute 16 iterations, grouped into four sets of four, where each set corresponds to one MemWr operation. In each iteration i, the forward dependency check (①) waits until the CiM array has completed the i-th computation ($i \geq$ Comp_cnt). For example, BufWr[8] in Fig. 8(b) cannot begin until CiMComp[8] completes. The backward dependency check (②) waits until the ODMAC has released the target OBUF bank ($i/$Tile_size $- 2 \geq$ MemWr_cnt), where the offset of 2 accounts for double buffering. For example, BufWr[0–3] fills OBUF[0], which must be fully transferred by MemWr[0] before BufWr[8–11] can reuse the same bank. After both conditions are met, the main_thread reads the output activation from the CiM array, writes it to the OBUF bank selected by the double-buffering index ($i/$Tile_size) $\bmod 2$, and signals completion (③). The same dependency logic applies to any valid tile count ratio accepted by Algorithm 1. The OBUF write latency is determined by the data-layout-aware memory model described in the next subsection.

The synchronization mechanism described above for the OAGU is not specific to this component. Any pipeline stage can be implemented as an independent thread with its own control loop, forward and backward dependency counters driven by shared completion signals, and a tile-count ratio that determines the dependency stride. The number and the direction of dependencies and the tile-count ratio are derived

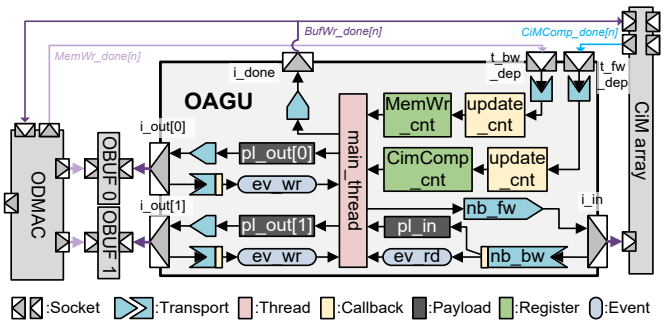
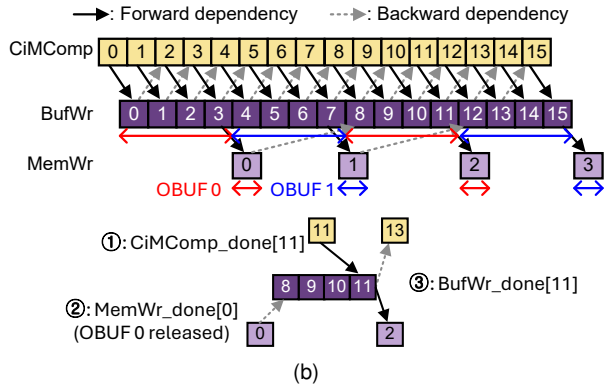


Fig. 7. Internal architecture of the OAGU.

```
void main_thread() {

  // OA_groups per tile (1 OA_group = M output pixels)
  int Tile_size = t_out_p * t_out_q;
  while (1) {
    // Process all P*Q OA_groups
    for (int i = 0; i < P * Q; i++) {
      // Wait for i-th computation to finish (per-OA_group)
①     wait(i >= CimComp_cnt);
      // Wait for (i/Tile_size-2)-th tile OBUF-to-Mem
      // write to finish (per-tile)
②     wait((i / Tile_size - 2) >= MemWr_cnt);
      // Read OAs from CIM array
      i_in.nb_fw(pl_in);
      // Wait for read completion
      wait(ev_rd);
      // Write OAs to OBUF, double buffering
      buf_index = (i / Tile_size) % 2;
      pl_out[buf_index] = pl_in;
      i_out[buf_index].nb_fw(pl_out[buf_index]);
      // Wait for buffer latency
      wait(ev_wr);
      // Signal buffer write completion
③     i_done.nb_fw(BufWr_done[i]);
} } }
```

(a)



(b)

Fig. 8. OAGU thread behavior. (a) Thread pseudocode. (b) Forward and backward dependencies (BufWr:MemWr = 4:1).

from the tiling produced by Algorithm 1. In CiMFlowSim, all five pipeline components in Fig. 6 are implemented using this mechanism. As boundary stages, MemRd has only a backward dependency, waiting for BufRd to release the IBUF, and MemWr has only a forward dependency, waiting for BufWr to fill the OBUF.

B. Data-Layout-Aware Memory Modeling

The memory data layout substantially affects the bandwidth/latency and energy consumption of memory access. Most of the analytical cost models used in the existing DSE approaches [18]–[20] simply rely on the data volume in estimating the latency/bandwidth and energy consumption of memory access, i.e., without taking into account the memory data layout, which turns out to limit the estimation accuracy severely, as will be shown later in this paper. This motivates us to propose the data-layout-aware memory modeling, which contributes to improving not only the estimation accuracy but also the optimality of Pareto front.

Fig. 9(a) shows the internal structure of one of the two OBUF components in Fig. 6. Each memory component exposes two target ports and dispatches incoming transactions through per-port handlers to a central arbiter. The arbiter maintains two persistent data structures: *tensor_registry* stores the static placement of each tensor, and *bank_state* tracks the most recent access time of each bank. Table II lists the fields maintained by these structures.

Fig. 9(b) shows the transaction sequence between the handler and the arbiter for a single tile access. When a pipeline stage issues a tile access through t_port[i], handler[i] receives the payload, creates a transaction containing the tensor identifier and the requested region, and submits it to the arbiter via a request event. The arbiter checks all pending requests and selects a single request (based on a configurable policy such as FIFO or priority) since OBUF is assumed to be single-port SRAM in the target accelerator system. It is worth mentioning that the proposed memory model is generic enough to model different types of SRAM such as dual-port SRAM. It then performs tile-to-line mapping using the tensor shape and layout from *tensor_registry*, translating the requested tile region into a set of physical line indices. Each line is fully accessed even when only a portion of the line contains valid tile data, which accounts for line underutilization. The arbiter waits for bank availability via *bank_state* and computes per-bank write or read latency depending on the transaction type. In both cases, the access latency depends on the most heavily loaded bank, whereas the access energy depends on the total number of accessed lines. The arbiter then updates *tensor_registry* and *bank_state*, annotates the transaction with the computed latency, and sends a grant event back to the handler. When multiple ports issue requests concurrently, the arbiter serializes them by selecting one request at a time. For example, when the OAGU in Fig. 7 writes to the OBUF through i_out[i], the OBUF arbiter in Fig. 9 computes the access latency and returns it via the ev_wr event, which the OAGU thread uses to advance the simulation time.

The memory model described above for the OBUF is instantiated for every memory component in Fig. 6, including both IBUF banks and the off-chip DRAM. All instances share the same arbiter logic, namely tensor registration, tile-to-line mapping, per-bank line counting, and max-loaded-bank latency computation. The instances differ only in configuration parameters such as clock frequency, base access latency, line width, and bank count. The registered tensor shapes also differ.

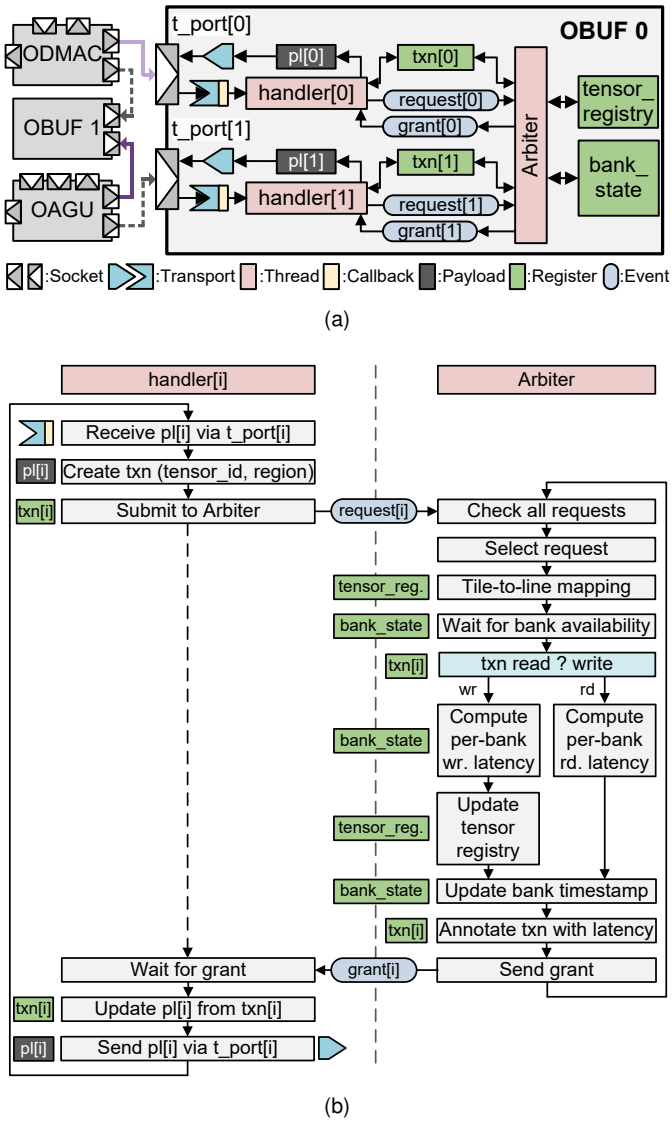


Fig. 9. Data-layout-aware memory model for OBUF. (a) Internal structure. (b) Transaction sequence.

TABLE II
ACCESS-MODEL STATE FIELDS

Field	Description
tensor_registry (per tensor)	
id	Tensor identifier
shape	Tensor dimensions
bitwidth	Bits per element
layout	Data layout
bank_range	Allocated bank indices
line_base	Starting physical line index
elements_per_line	Tensor elements packed per physical line
bank_state (per bank)	
last_access_time	Most recent access completion time

On-chip buffers register tile-sized tensors determined by the tiling, whereas DRAM registers full activation maps. The access logic resides entirely in the arbiter, and reconfiguring a memory component requires only a different set of parameters.

TABLE III
EVALUATED CIM ACCELERATOR HARDWARE PARAMETERS

Parameter	MRAM [6]	eFlash [5]
Process Node	22 nm	65 nm
Macro Array Size (I×O)	576×72	32×64
Compute Latency	20 ns	50 ns
Energy Efficiency	46 TOPS/W	333 TOPS/W

V. EXPERIMENTAL RESULTS

This section evaluates the estimation accuracy of CiM-FlowSim and the impact of decoupled loop tiling on DSE results. Experiments are conducted on two NVM-based CiM accelerators whose architectural parameters are summarized in Table III. Both accelerators use LPDDR4-3200 off-chip memory with a bandwidth of 6.4 GB/s. Four convolutional neural network (CNN) benchmarks are evaluated: LeNet-5 [36], VGG-11 [37], MobileNetV1 [38], and ResNet-18 [39]. The MobileNetV1 evaluation excludes depthwise layers, as depthwise convolutions are inefficient under the weight-stationary dataflow assumed in this work. The analytical cost model used as the baseline is based on MAESTRO [18] and is adjusted to match the target hardware in Table III.

For a network with N convolutional layers, the total latency and energy are $L_{\text{net}} = \sum_{\ell=1}^{N} L_\ell$ and $E_{\text{net}} = \sum_{\ell=1}^{N} E_\ell$, whereas the required on-chip buffer area is $A_{\text{net}} = \max_\ell A_\ell$. The two DSE objectives are latency and the energy–area product, $\text{EAP}_{\text{net}} = E_{\text{net}} \times A_{\text{net}}$, where E_{net} sums the energy consumed in off-chip DRAM, the on-chip buffers, and the CiM array. In terms of area, tiling decisions affect only the buffer requirements. Therefore, A_{net} denotes only the required on-chip buffer area, as all other hardware blocks remain fixed. Since the number of all per-layer tiling combinations grows exponentially with the number of layers, the network-level Pareto front is approximated by randomly sampling 100 million tiling combinations, where each sample selects one valid tiling per layer and computes the network-level latency and EAP. The sample count was determined empirically, as further increases did not change the resulting HV rankings.

Pareto front quality is measured by HV [24]. Each Pareto-optimal point dominates the rectangular region between itself and a reference point, and HV is the union area of all such regions. A larger HV indicates a Pareto front that covers a wider range of the objective space with better tradeoffs. In this work, HV is computed in the two-dimensional space of (latency, EAP). The reference point for each hardware-network pair is set to 5% above the worst latency and EAP among the Pareto-optimal points across the compared methods. HV is normalized by the rectangular area formed by the reference point and the point $(L_{\text{best}}, \text{EAP}_{\text{best}})$, where L_{best} and EAP_{best} are the best observed latency and EAP independently across all Pareto-optimal points.

A. Estimation Error Analysis

This subsection analyzes the estimation errors of the analytical cost model and their impact on DSE results.

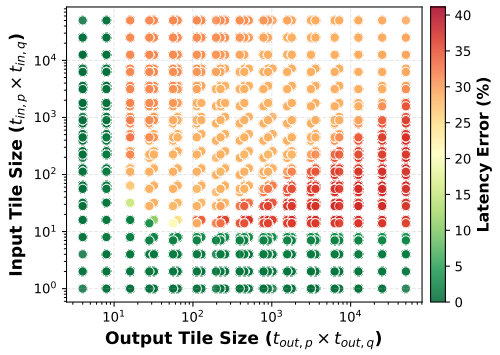


Fig. 10. Latency estimation error over the two-dimensional tile-size space (MRAM, VGG-11, Layer 0).

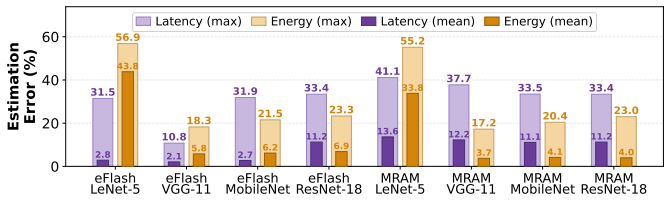


Fig. 11. Analytical latency and energy estimation errors across all hardware-network pairs. Thin bars denote mean relative error. Wide bars denote maximum relative error.

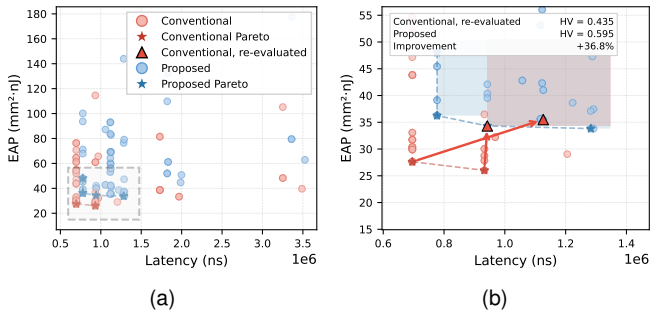


Fig. 12. Coupled loop tilings evaluated analytically and with CiMFlowSim (MRAM, LeNet-5). (a) All network-level points. (b) Pareto region (detail). Arrows show re-evaluated analytical Pareto points.

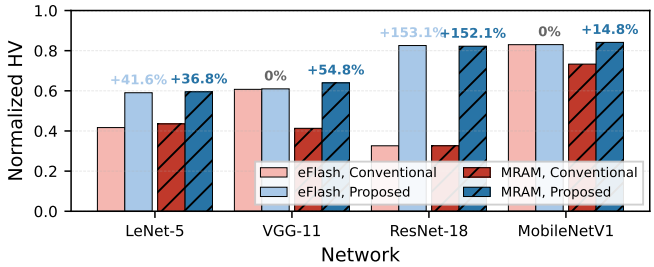


Fig. 13. Normalized hypervolume of coupled loop tilings across all evaluated hardware-network pairs: Analytical vs. CiMFlowSim.

Fig. 10 shows the latency estimation error of the analytical cost model relative to CiMFlowSim over the two-dimensional space of input and output tile sizes for (MRAM, VGG-11, Layer 0). The error ranges from near zero to 37.7% across tilings, indicating that it is tiling-dependent rather than a uniform bias. Low-error and high-error regions are scattered across the tile-size space, and no monotonic trend is observed along either axis. This tiling-dependent error is unlikely to be corrected by applying a single calibration factor to the analytical cost model and may cause the DSE to end up with non-optimal design points.

Fig. 11 shows the latency and energy estimation errors of the analytical cost model relative to CiMFlowSim across all hardware-network pairs. The deviations are observed in every case, with maximum latency errors ranging from 10.8% to 41.1% and maximum energy errors from 18.3% to 56.9%. The large gap between mean and maximum values confirms that the tiling-dependent error observed in Fig. 10 persists across all evaluated configurations.

To assess whether these estimation errors affect network-level DSE results, the Pareto fronts obtained by the analytical baseline and CiMFlowSim are compared. Fig. 12 shows all coupled loop tilings for (MRAM, LeNet-5) evaluated by both methods at the network level, where blue circles represent CiMFlowSim and red circles represent the analytical baseline. In Fig. 12(a), the analytical Pareto-optimal points (red stars) appear closer to the origin than the CiMFlowSim Pareto-optimal points (blue stars), suggesting lower latency and lower EAP. Since the analytical cost model suffers from the estimation errors shown above, this apparent advantage may not be reliable. In Fig. 12(b), the Pareto region is shown in detail, and the analytical Pareto-optimal points are re-evaluated with CiMFlowSim (red triangles). The re-evaluated points shift toward higher latency and higher EAP, confirming that the analytical cost model underestimates the latency and the EAP of these design points. As a result of this shift, rank reversals occur, and design points with lower latency and EAP may be missed. HV improves from 0.435 to 0.595 (+36.8%) when the Pareto front is re-evaluated with CiMFlowSim.

Fig. 13 reports normalized HV across all evaluated hardware-network pairs. CiMFlowSim improves HV by up to 153.1% (eFlash, ResNet-18). The largest improvement occurs for ResNet-18, where the greater number of layers increases the likelihood of per-layer estimation errors affecting the network-level Pareto front. By contrast, the HV difference is near zero for (eFlash, VGG-11) and (eFlash, MobileNetV1). Although the analytical cost model still exhibits errors for these cases (Fig. 11), small tiles do not increase the pipeline latency when the compute stage is dominant. Therefore, the selected tilings have low estimation error, reducing the chance of rank reversals.

B. Loop Tiling Optimization

The previous subsection has shown the need for an accurate cost model. This subsection evaluates the effect of expanding the tiling design space through decoupled loop tiling.

Fig. 14(a) and (b) show all coupled (blue) and decoupled (green) loop tilings for Layer 0 and Layer 1 of (MRAM, LeNet-5), respectively, in the latency–EAP space. At the per-layer level, the decoupled loop tilings extend the design space beyond the coupled loop tilings, though the effect

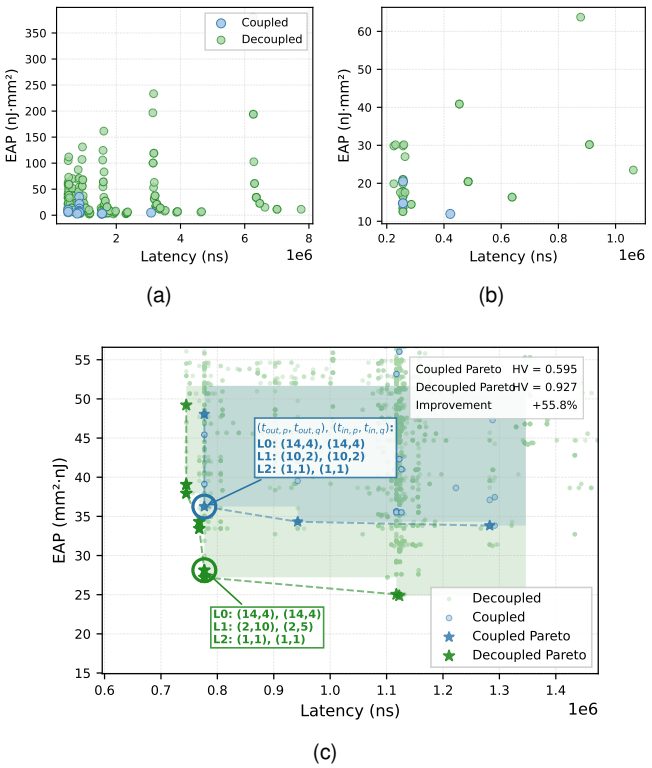


Fig. 14. Effect of decoupled loop tiling (MRAM, LeNet-5) in the latency–EAP space. (a) Layer 0. (b) Layer 1. (c) Network-level.

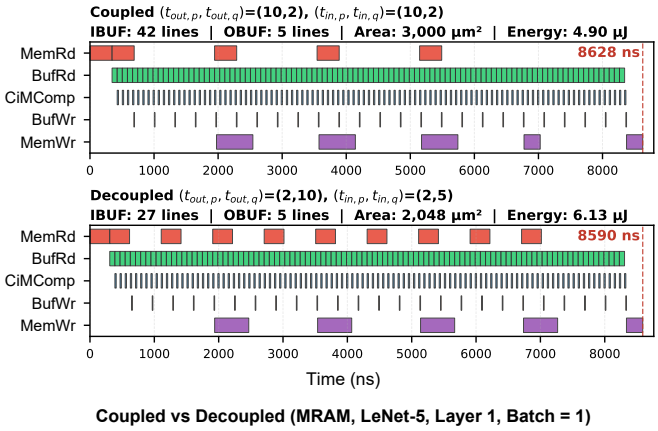


Fig. 15. Cycle traces of coupled and decoupled design points at nearly identical latency (MRAM, LeNet-5, Layer 1).

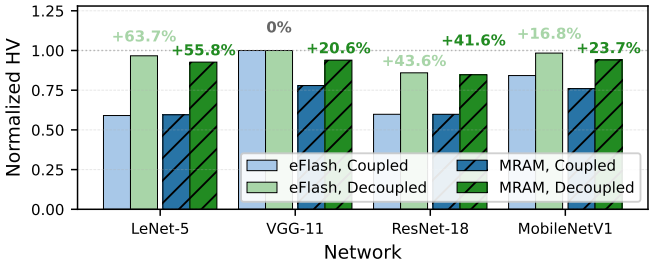


Fig. 16. Normalized hypervolume across all evaluated hardware-network pairs. Coupled vs. decoupled loop tiling. Percentages denote HV gain from decoupled loop tiling.

on the per-layer Pareto front appears limited. At the network level, latency and energy are accumulated across layers while buffer area is determined by the single largest layer ($A_{\text{net}} = \max_\ell A_\ell$). Therefore, the network-level Pareto front depends on the combination of per-layer tiling selections, and a wider range of tradeoffs can be achieved than any single layer suggests. Fig. 14(c) compares the resulting network-level Pareto fronts. The inclusion of decoupled loop tiling increases HV from 0.595 to 0.927 (+55.8%) by introducing design points with lower EAP. Under decoupled loop tiling, the input and output tile sizes are set independently, allowing the most area-demanding layer to reduce its buffer requirement without increasing latency. As a result, the Pareto front is extended into a region not covered by coupled loop tiling.

Two design points with nearly identical latency are selected from the coupled and decoupled Pareto fronts in Fig. 14(c), marked with circles. These two points differ only in the Layer 1 tiling. Fig. 15 compares the cycle traces of Layer 1 for these two points. In the decoupled case, a smaller input tile is used, which requires fewer IBUF lines and reduces the buffer area by 32%. However, the decoupled tiling incurs more MemRd iterations owing to redundant reads of overlapping halo regions, increasing the energy by 25%. The additional MemRd iterations overlap with the bottleneck stage, and the overall latency is not increased. As a result, the per-layer energy increase has a smaller impact on the network-level EAP than the buffer area reduction.

Whether the HV gain holds across all hardware-network pairs is examined next. Fig. 16 reports the results for all evaluated pairs. The maximum HV gain of 63.7% is observed for (eFlash, LeNet-5). By contrast, the HV gain for (eFlash, VGG-11) is near zero. When the compute stage dominates the pipeline latency, the memory stages have a reduced impact on the overall latency, leaving limited room for tile sizing to improve the Pareto front.

VI. Conclusion

In this paper, CiMFlowSim, a cycle-level dataflow simulator for weight-stationary CiM accelerators, has been presented. The proposed simulator employs a dependency-aware pipeline model to capture inter-stage pipeline stalls and a data-layout-aware memory model to account for memory data layout overheads. In addition, decoupled loop tiling is incorporated into the weight-stationary CiM pipeline so that the input and output tile sizes can be configured independently, thereby expanding the design space beyond conventional coupled tiling. Experimental results show that the estimation error of the analytical cost model amounts to 41.1% in latency and 56.9% in energy. These tiling-dependent errors cannot be corrected by a single calibration factor and may cause the DSE to end up with non-optimal design points. When applied to loop tiling optimization, CiMFlowSim improves the hypervolume by up to 153.1%. In addition, incorporating decoupled loop tiling into the design space leads to the HV gain of up to 63.7%. These results demonstrate that both cost-model accuracy and

design-space coverage are of primary importance in the DSE of CiM accelerators.

REFERENCES

[1] J. Wang, S. Kim, J. Heo, and C. S. Park, "eF2lowSim: System-level simulator of eFlash-based compute-in-memory accelerators for convolutional neural networks," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2023, pp. 1–6.

[2] A. A. Khan, J. P. C. De Lima, H. Farzaneh, and J. Castrillon, "The landscape of compute-near-memory and compute-in-memory: A research and commercial overview," *arXiv preprint arXiv:2401.14428*, 2024.

[3] W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu *et al.*, "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, pp. 504–512, 2022.

[4] S. Jung, H. Lee, S. Myung, H. Kim, S. K. Yoon, S.-W. Kwon, Y. Ju, M. Kim, W. Yi, S. Han *et al.*, "A crossbar array of magnetoresistive memory devices for in-memory computing," *Nature*, vol. 601, pp. 211–216, 2022.

[5] E. Choi, I. Choi, V. Lukito, D.-H. Choi, D. Yi, I.-J. Chang *et al.*, "A 333TOPS/W logic-compatible multi-level embedded flash compute-in-memory macro with dual-slope computation," in *2023 IEEE Custom Integrated Circuits Conference (CICC)*, 2023, pp. 1–2.

[6] Y.-C. Chiu, W.-S. Khwa, C.-Y. Li, F.-L. Hsieh, Y.-A. Chien, G.-Y. Lin *et al.*, "A 22nm 8Mb STT-MRAM near-memory-computing macro with 8b-precision and 46.4-160.1TOPS/W for edge-AI devices," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 496–497.

[7] W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu *et al.*, "A compute-in-memory chip based on resistive random-access memory," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 4, pp. 1055–1069, 2022.

[8] W.-H. Huang, T.-H. Wen, J.-M. Hung, W.-S. Khwa, Y.-C. Lo, C.-J. Jhang *et al.*, "A nonvolatile AI-edge processor with 4MB SLC-MLC hybrid-mode ReRAM compute-in-memory macro and 51.4-251 TOPS/W," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 258–259.

[9] T.-H. Wen, H.-H. Hsu, W.-S. Khwa, W.-H. Huang *et al.*, "A 22nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, 2024, pp. 580–582.

[10] J. M. Correll, V. Bothra, F. Cai, Y. Lim, S. H. Lee, S. Lee *et al.*, "A fully integrated reprogrammable CMOS-RRAM compute-in-memory coprocessor for neuromorphic applications," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 6, no. 1, pp. 36–44, 2020.

[11] C. Song and D. S. Jeong, "Computing-in-memory dataflow for minimal buffer traffic," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 209–216.

[12] L. Han, P. Huang, Z. Zhou, Y. Chen, X. Liu, and J. Kang, "A convolution neural network accelerator design with weight mapping and pipeline optimization," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.

[13] K. He, I. Chakraborty, C. Wang, and K. Roy, "Design space and memory technology co-exploration for in-memory computing based machine learning accelerators," in *2022 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2022, pp. 1–9.

[14] S. Kim, J. Heo, H. Shin, J. Lee, S. Park, and C. S. Park, "NetTLM-DSE: Design space exploration for DNN layer-pipeline spatial mappings," in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2026, pp. 889–895.

[15] C. Park, S. Park, and C. S. Park, "Roofline-model-based design space exploration for dataflow techniques of CNN accelerators," *IEEE Access*, vol. 8, pp. 172 509–172 523, 2020.

[16] B. Carrion Schafer and Z. Wang, "High-level synthesis design space exploration: Past, present, and future," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2628–2639, 2020.

[17] C. S. Park and S. Park, "CNN accelerator performance dependence on loop tiling and the optimum resource-constrained loop tiling," *IEEE Access*, vol. 13, pp. 16 800–16 810, 2025.

[18] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflows: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2019, pp. 754–768.

[19] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara *et al.*, "Timeloop: A systematic approach to DNN accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[20] S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang, "TileFlow: A framework for modeling fusion dataflow via tree-based analysis," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 1271–1288.

[21] L. Mei, P. Houshmand, V. Jain, J. S. Piedrahita Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1160–1174, 2021.

[22] C. Wang, W. Pang, X. Wu, G. Jun, L. Romero, E. Taka *et al.*, "Can asymmetric tile buffering be beneficial?" *arXiv preprint arXiv:2511.16041*, 2025.

[23] X. Peng, S. Huang, H. Jiang, A. Lu, and S. Yu, "DNN+NeuroSim v2.0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 11, pp. 2306–2319, 2021.

[24] P. Xu, S. Zheng, M. Wang, Z. Yu, S. Chen, T. Chen *et al.*, "Rank-DSE: Neural pareto comparator of microarchitecture design space exploration," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 5, pp. 71:1–71:24, 2025.

[25] M. Pellauer, Y. S. Shao, J. Clemons, N. C. Crago, K. Hegde, R. Venkatesan *et al.*, "Buffets: An efficient and composable storage idiom for explicit decoupled data orchestration," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 137–151.

[26] L. P. Carloni, K. L. McMillan, and A. L. Sangiovanni-Vincentelli, "Theory of latency-insensitive design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 9, pp. 1059–1076, 2001.

[27] J. Wang, M. Ge, B. Ding, Q. Xu, S. Chen, and Y. Kang, "NicePIM: Design space exploration for processing-in-memory DNN accelerators with 3-D stacked-DRAM," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1456–1469, 2024.

[28] J. Tong, A. Itagi, P. Chatarasi, and T. Krishna, "FEATHER: A reconfigurable accelerator with data reordering support for low-cost on-chip dataflow switching," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 198–214.

[29] J. Wang, S. Park, and C. S. Park, "Optimization of communication schemes for DMA-controlled accelerators," *IEEE Access*, vol. 9, pp. 139 228–139 247, 2021.

[30] IEEE, "IEEE standard for standard SystemC language reference manual," 2023, IEEE Std 1666-2023.

[31] Doulos, "TLM-2.0 user manual," 2008, oSCI TLM-2.0 Language Reference Manual.

[32] S. Kim, J. Wang, Y. Seo, S. Lee, Y. Park, S. Park *et al.*, "Transaction-level model simulator for communication-limited accelerators," *arXiv preprint arXiv:2007.14897*, 2020.

[33] J. Wang, Y. Gim, S. Park, and C. S. Park, "Modeling and simulation of system bus and memory collisions in heterogeneous SoCs," *IEEE Access*, vol. 10, pp. 25 901–25 921, 2022.

[34] J. Wang, S. Park, and C. S. Park, "Spatial data dependence graph based pre-RTL simulator for convolutional neural network dataflows," *IEEE Access*, vol. 10, pp. 11 382–11 403, 2022.

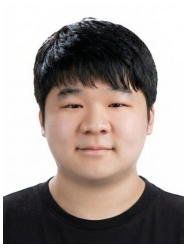
[35] S. Kim, S. Park, and C. S. Park, "System-level communication performance estimation for DMA-controlled accelerators," *IEEE Access*, vol. 9, pp. 141 389–141 402, 2021.

[36] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[37] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556.

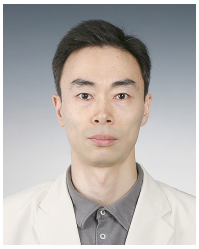
[38] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.



Junsu Heo received the B.S. and M.S. degrees in electrical and electronics engineering from Konkuk University, Seoul, South Korea, where he is currently pursuing the Ph.D. degree.

His research interests include SoC architecture design, virtual-prototype-based system simulation, and algorithm-hardware co-optimization.



Sungkyung Park (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees all from Seoul National University, South Korea.

From 2002 to 2004, he was with Samsung Electronics, Suwon, Korea, as a Senior Engineer, where he worked on the development of system-level simulators for cellular standards. From 2004 to 2006, he was with Electronics and Telecommunications Research Institute (ETRI), Daejeon, Korea, as a Senior Member of Research Staff, where he worked on the fiber-optic front-end IC design. From 2006 to 2009, he was with Ericsson Inc., as a Senior Staff Hardware Designer, where he worked on the design and modeling of multi-standard RF transceivers and clocking circuits. In 2009, he joined the Faculty of the Department of Electrical and Electronics Engineering, Pusan National University, Busan, South Korea, where he is currently a Professor.

His research interests include SoC modeling and design, hardware accelerator and processor design, and virtual platform development for AI and wireless communication.

Chester Sungchung Park (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronics engineering all from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea.

From 2006 to 2007, he was with Samsung Electronics, Giheung, South Korea, as a Senior Engineer, where he worked on flash memory controller including error correction codes. From 2007 to 2013, he was with Ericsson Research, CAL, USA, as a Senior Engineer, where he worked on wireless communication algorithms, circuit modeling, and the relevant 3GPP standardizations. Since 2013, he has been with the Department of Electrical and Electronics Engineering, Konkuk University, Seoul, South Korea, where he is currently working on algorithm-architecture co-optimizations using virtual prototype based system simulations.